

LLM

Семинар 10

План

- Постановка задачи
- Tokenизация
- Эмбединги
- Attention
- Собираем все вместе



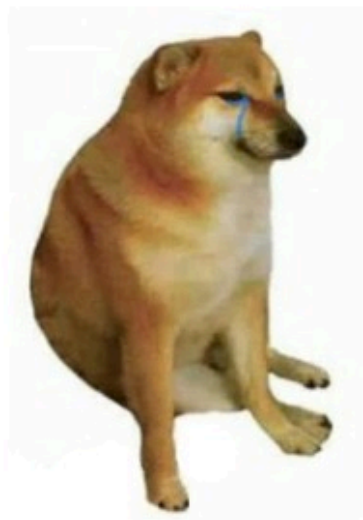
Чем же на самом деле являются LLM?

Чем же на самом деле являются LLM?

T9.

По сути LLM угадывают, какое следующее слово должно идти за уже имеющимся текстом

T9



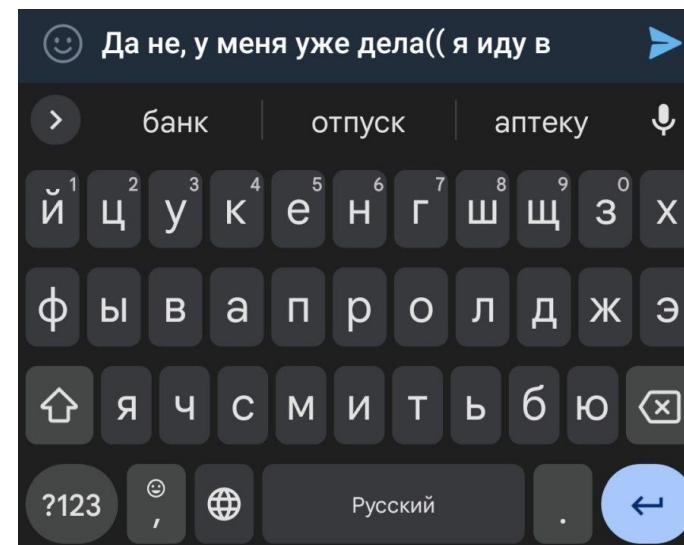
Набрал «милая, ты допа»?
Хм-м, исправлю-ка
на «ты жопа»!

imgflip.com

ChatGPT



Хочешь инструкцию
к пылесосу в стихах в
стиле Тютчева? Ща, пададжи



Постановка задачи

T9 на клавиатуре смартфона, и ChatGPT обучены решать до безумия простую задачу: **предсказание единственного следующего слова**. Это и есть языковое моделирование – когда по некоторому уже имеющемуся тексту делается вывод о том, что должно быть написано дальше.

Постановка задачи

Примеры задач:

- **Машинный перевод**

Русский <---> Английский

- **Суммаризация**

Длинный текст → короткий текст

- **Изменение стиля текста**

Грубый <---> Вежливый

Формальный <---> Неформальный

- **Ответ на вопрос**

Вопрос -> Ответ

Токенизация

Наши модели **не оперируют словами**.

Они оперируют **токенами**.

Токен – это минимальная смысловая единица текста (слово, часть слова, символ или знак препинания), на которые модель разбивает входные данные для анализа

Токенизация

Модели работают не с текстом напрямую, а с числами, поэтому каждый токен преобразуется в уникальное числовое значение — токен ID из словаря модели

Например

Текст: *«Привет, мир!»*

Токены: *["Привет", ",", " ", "мир", "!"]*

Список ID токенов: *[1234, 5678, 452, 9101, 1121].*

Токенизация

Подход	Пример	Словарь	Проблема
По словам	«Собака гавкает, кошка прячется.» -> ["Собака", " ", "гавкает", ",", " ", "кошка", " ", "прячется", "."]	Очень большой	Невозможно охватить все падежи, опечатки и т.д.
По символам	«Собака» -> ["С", "о", "б", "а", "к", "а"]	Маленький	Теряется смысл, последовательность слишком длинная
Subword tokenization (подсловная токенизация)	«Собака гавкает, кошка прячется.» -> ["Собак", "а", " гавк", "ает", ",", " кошк", "а", " пряч", "ется", "."]	Что-то среднее	Золотая середина

Токенизация

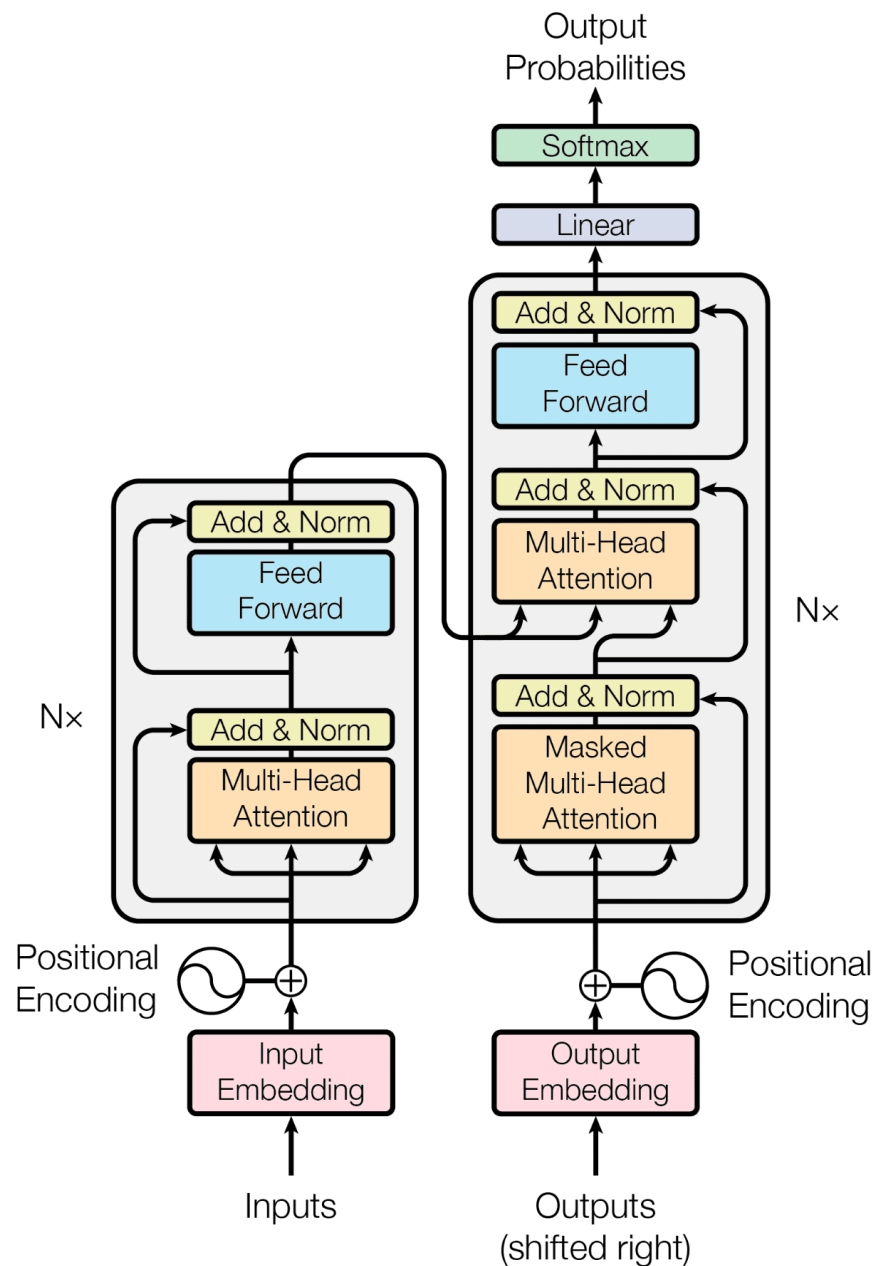
ноутбук

Transformer

Attention is all you need

- Самая популярная архитектура с 2017 года
- ~200k цитат у оригинальной статьи

Разберемся с тем, как она работает



Эмбе́ддинги

Мы уже проходили как получать эмбе́ддинги через TF-IDF и BoW, но в LLM эмбе́ддинги строятся чуть по другому.

Каждому токену из словаря ставится в соответствие обучаемый вектор фиксированной размерности `d_model` (обычно 768, 1024, 4096, 8192 и т.д.). Эти векторы хранятся в матрице эмбе́ддингов размером `vocab_size × d_model`. На этапе обучения матрица обновляется градиентным спуском.

* Рисуем на доске

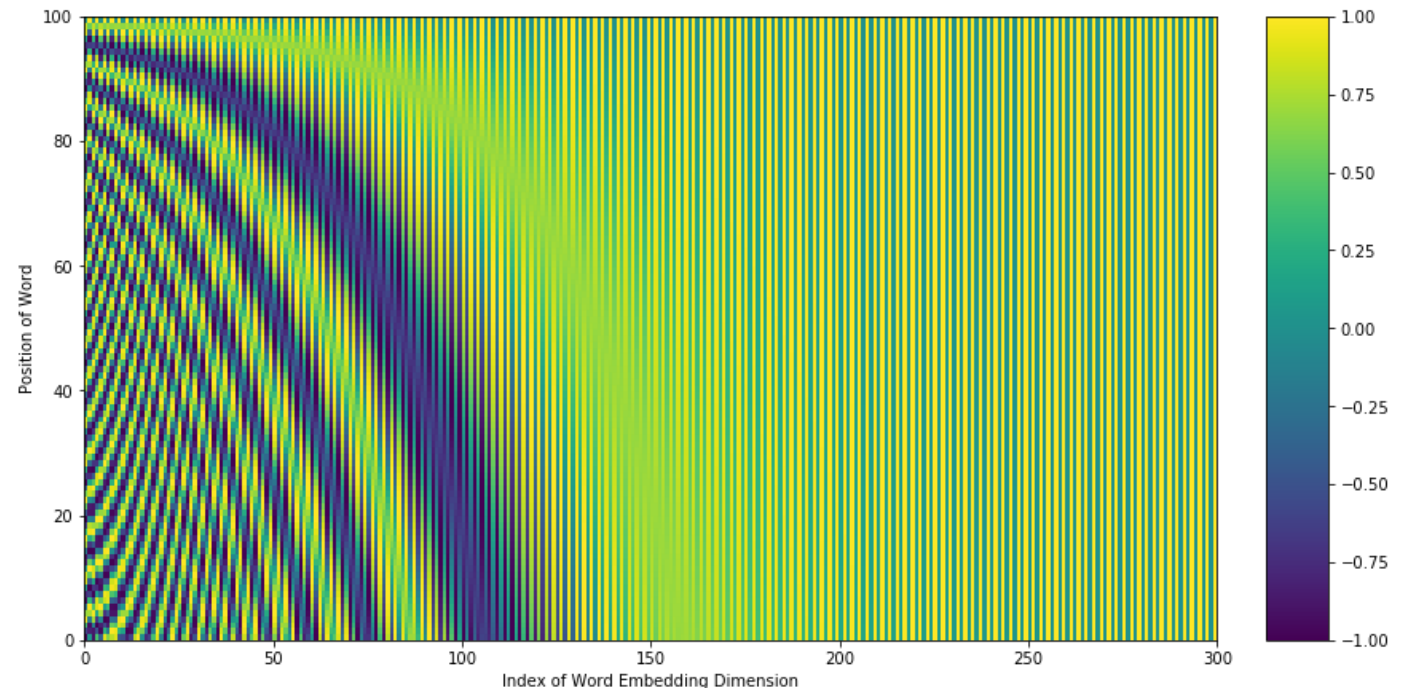
Эмбе́ддинги

В LLM нет встроенного понятия порядка слов. Поэтому нужно добавить информацию о позиции токена в последовательности.

Вариант 1: фиксированные синусоидальные функции

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Эмбе́ддинги

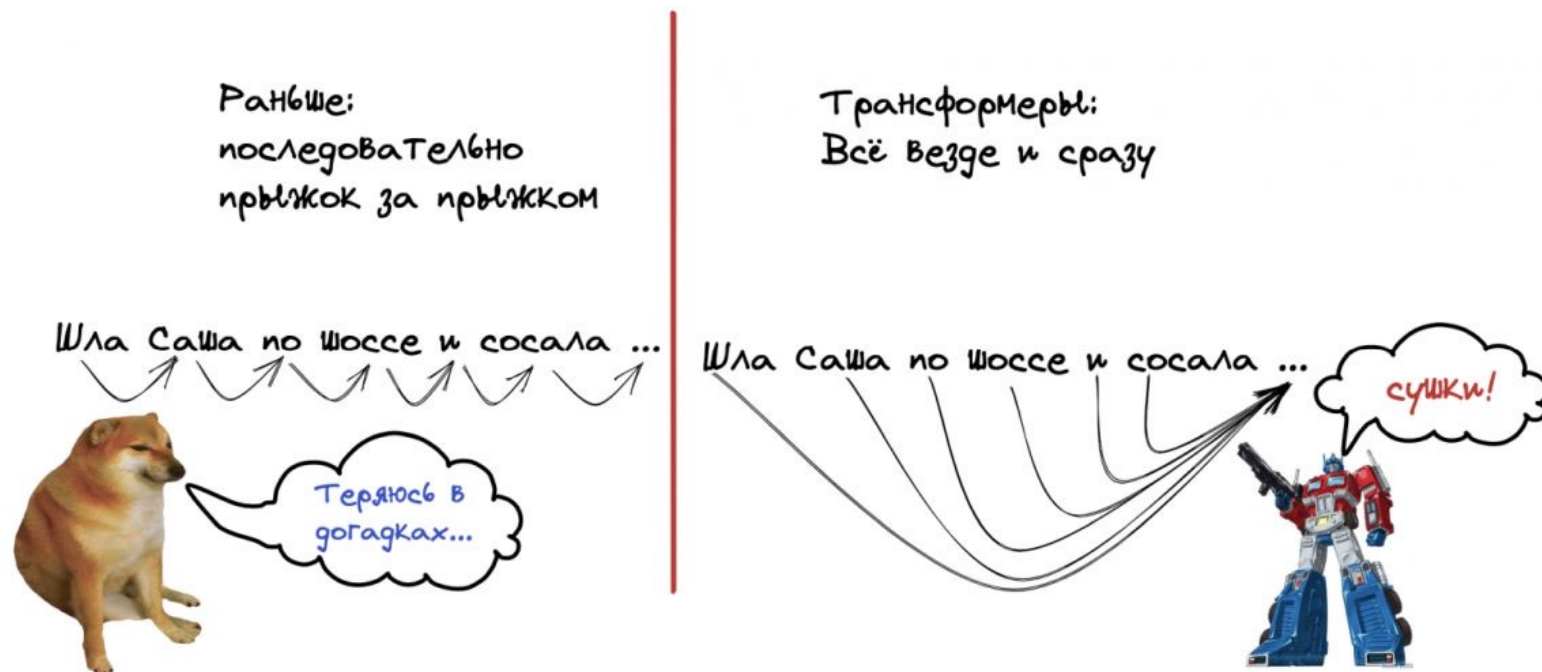
В LLM нет встроенного понятия порядка слов. Поэтому нужно добавить информацию о позиции токена в последовательности.

Вариант 2: обучаемые позиционные эмбе́ддинги

Создается матрица $\text{max_seq_len} \times \text{d_model}$, и каждая позиция (0, 1, 2, ...) имеет свой обучаемый вектор, который складывается с token embedding.

Attention

Новизна Transformer заключалась в использовании self-attention — механизма, благодаря которому модель может сосредоточиться на наиболее важных элементах входной последовательности



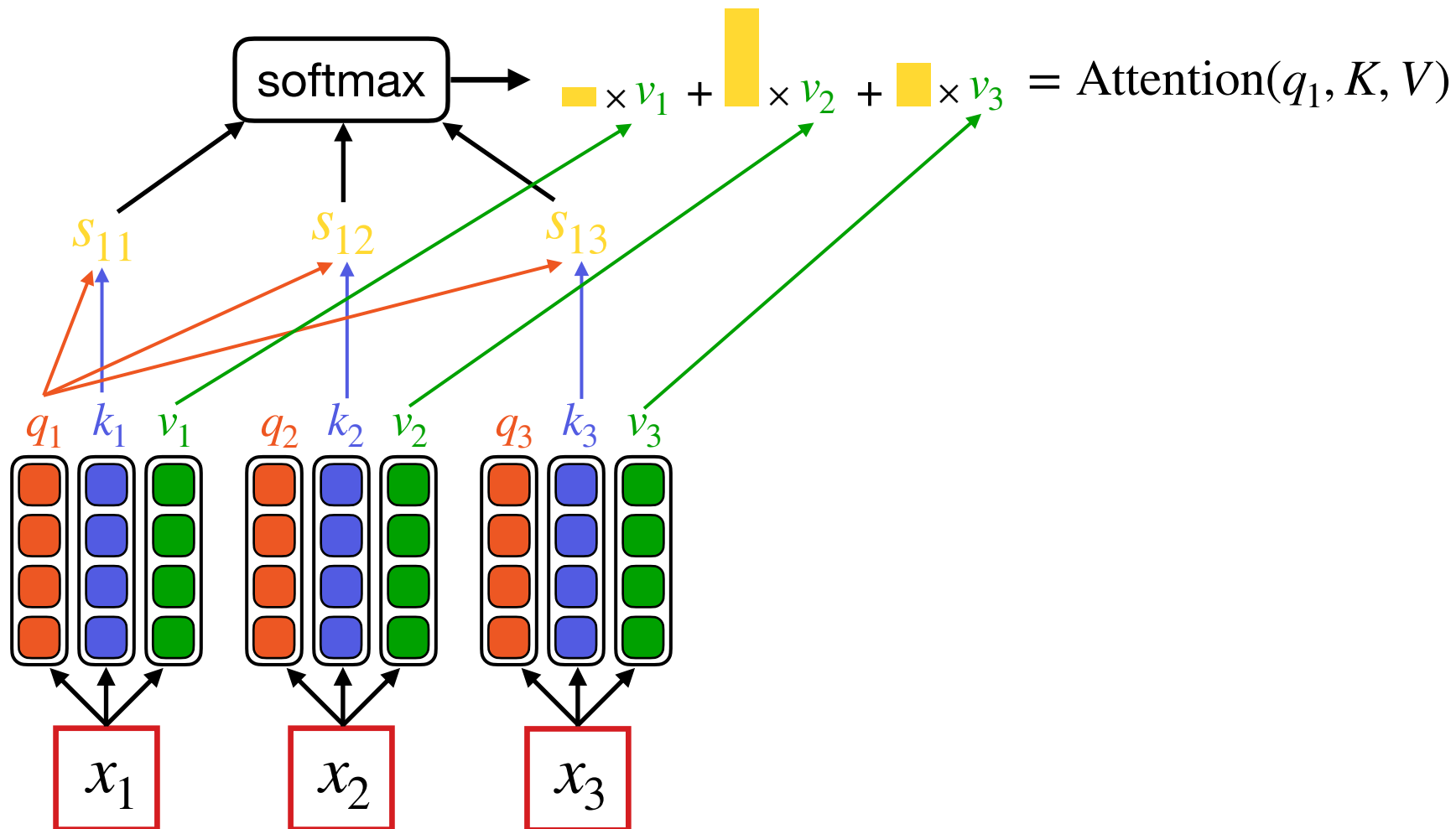
Self-attention

Позволяет каждому токenu смотреть на все остальные.

q – запрос (query)

k – ключ (key)

v – значение (value)



Self-attention

Позволяет каждому токenu смотреть на все остальные.

Query: $Q = W_q x + b_q$

Key: $K = W_k x + b_k$

Value: $V = W_v x + b_v$

$$x \in \mathbb{R}^{n \times d}$$

$$W_q, W_k, W_v \in \mathbb{R}^{d \times d}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

нормировочная
константа



Self-attention

Позволяет каждому токенту смотреть на все остальные.

Query: $Q = W_q x + b_q$

Key: $K = W_k x + b_k$

Value: $V = W_v x + b_v$

$$x \in \mathbb{R}^{n \times d}$$

$$W_q, W_k, W_v \in \mathbb{R}^{d \times d}$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

нормировочная
константа



Attention

Each vector receives three representations (“roles”)

$$\begin{bmatrix} W_Q \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$$

Query: vector **from** which the attention is looking

“Hey there, do you have this information?”

$$\begin{bmatrix} W_K \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$$

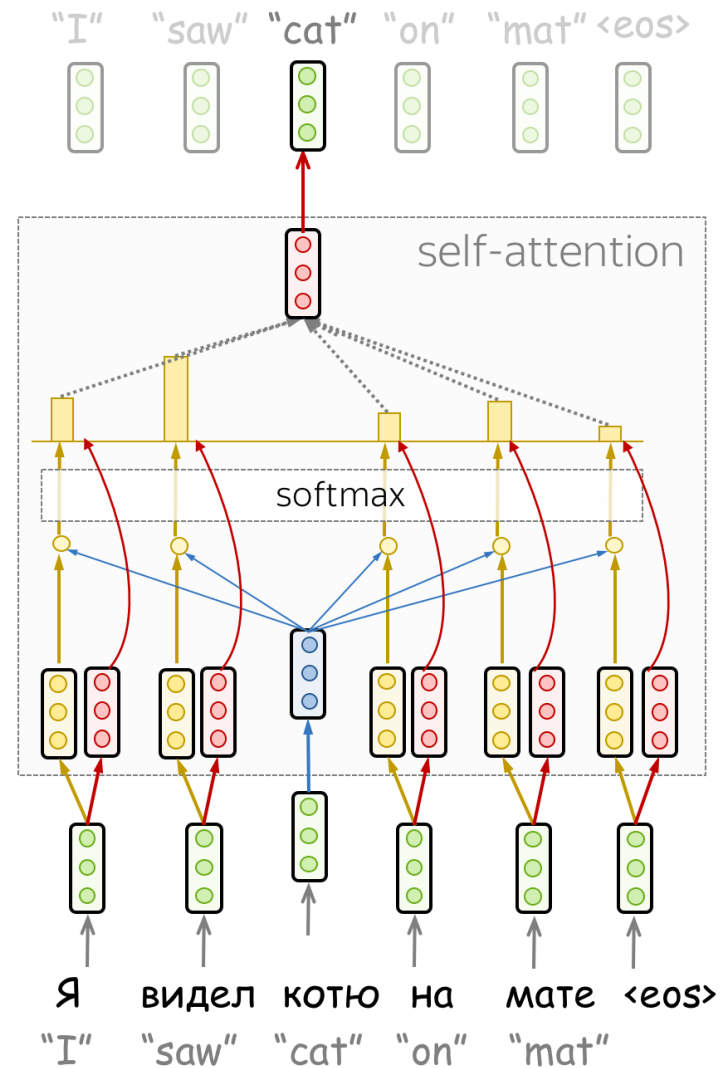
Key: vector **at** which the query looks to compute weights

“Hi, I have this information – give me a large weight!”

$$\begin{bmatrix} W_V \end{bmatrix} \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$$

Value: their weighted sum is attention output

“Here’s the information I have!”



Multi-Head Self-attention

Self-attention позволяет запрашивать только информацию одного вида.
А что если мы хотим узнать разные аспекты?

На улице ярко светит солнце

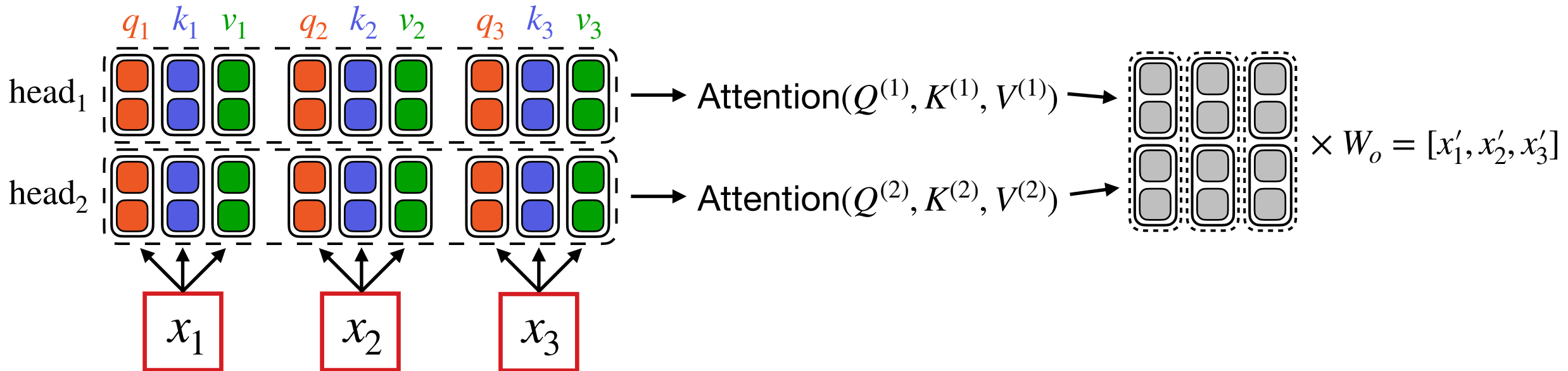


как? что?

Разделим внимание на несколько голов

Multi-Head Self-attention

- Делим каждый вектор q , k , v на равные части
- Считаем self-attention для каждой части отдельно
- Конкатенируем результат
- Домножаем на выходную матрицу



Multi-Head Self-attention

Query: $Q^{(i)} = W_q^{(i)}x + b_q^{(i)}$

Key: $K^{(i)} = W_k^{(i)}x + b_k^{(i)}$

Value: $V^{(i)} = W_v^{(i)}x + b_v^{(i)}$

$$\text{head}^{(i)} = \text{Attention}(Q^{(i)}, K^{(i)}, V^{(i)})$$

$$\text{MultiHead}(Q, K, V) = W_o \times [\text{head}^{(1)}, \dots, \text{head}^{(h)}]$$

$$W_q^{(i)}, W_k^{(i)}, W_v^{(i)} \in \mathbb{R}^{d_{\text{head}} \times d}$$

$$W_o \in \mathbb{R}^{d \times h \cdot d_{\text{head}}}$$

Методы семплирования токенов

Модель выдает распределение вероятностей токенов



Как выбрать токен из этого распределения?

Жадное семплирование

Greedy Sampling

Можно всегда выбирать токен с максимальной вероятностью

$$y_t = \operatorname{argmax}_{y \in V} p(y | y_{<t})$$

Плюсы:

- Максимизируем вероятность текста

Минусы:

- Теряем разнообразие текста

Beam Search

Лучевой поиск

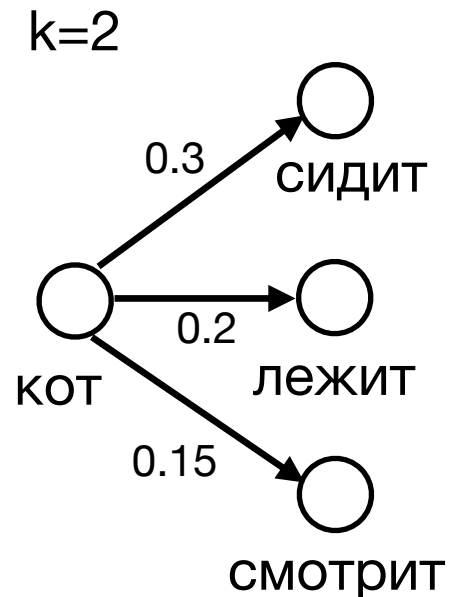
- Попробуем максимизировать вероятность текста еще больше
- При жадном семплировании мы **не учитываем** влияние токена на следующие за ним
- Будем строить предсказания на несколько токенов вперед, а затем выбирать токен с наибольшей совокупной вероятностью

Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

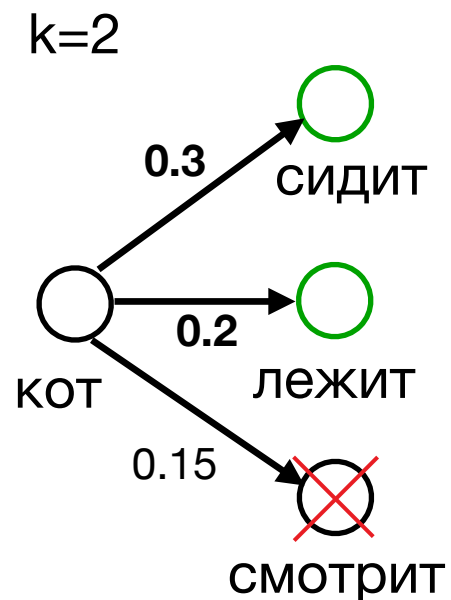


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

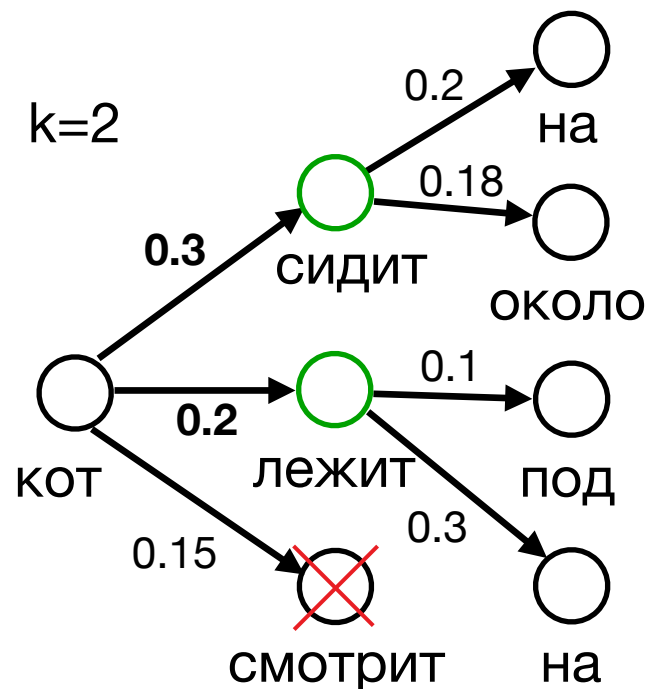


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

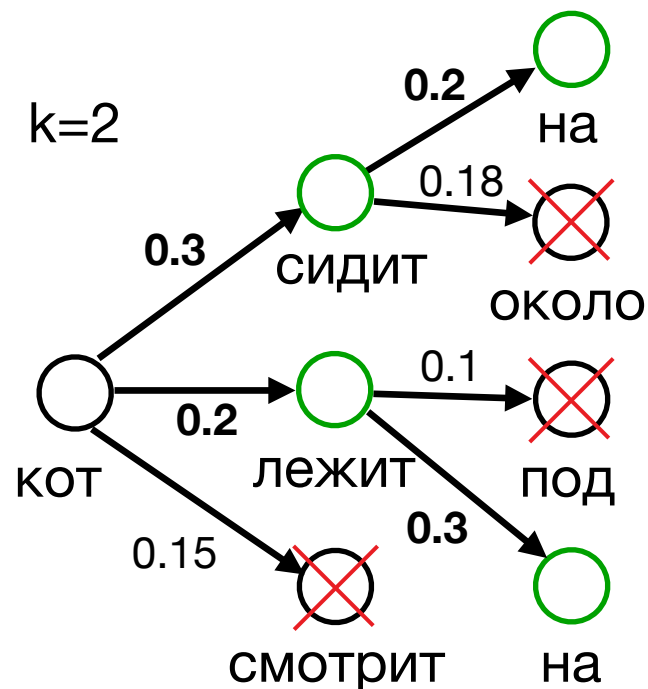


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

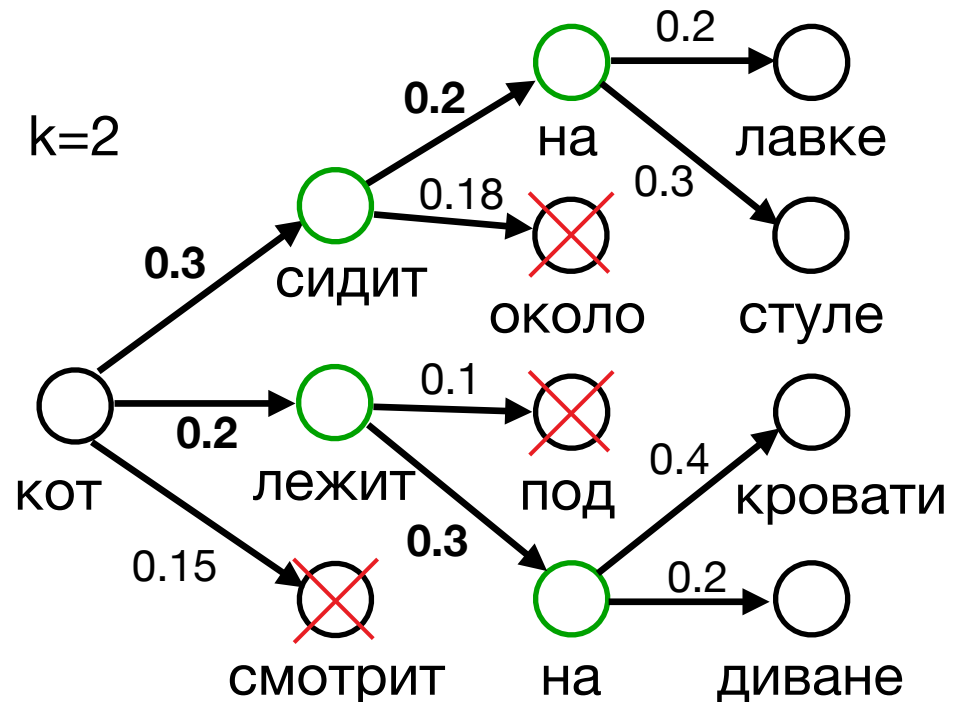


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных

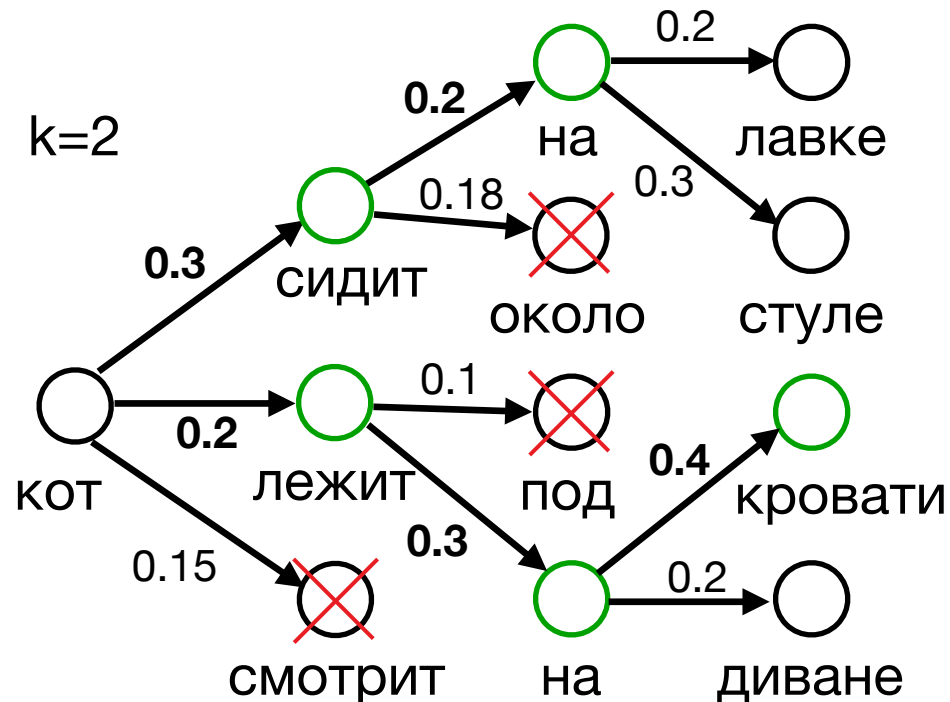


Beam Search

Лучевой поиск

Идея: Поддерживаем k наиболее вероятных траекторий

- На каждом шаге генерируем k продолжений для каждой траектории
- Оставляем только k самых вероятных



Траектория с "лежит" имеет максимальную вероятность

=> выбираем "лежит"

Beam Search vs Жадное семплирование

- Beam Search работает лучше для всех seq2seq задач
- Beam Search работает гораздо медленнее (зависит от k)
- Популярные значения k – 3 или 5

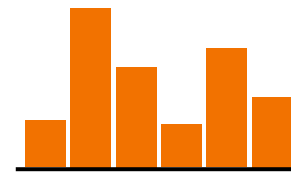
Семплирование с температурой

- При обычном семплировании с вероятностями вероятностная масса случайных токенов слишком велика
- Сделаем распределение более вырожденным, добавив температуру $\tau \in [0,1]$

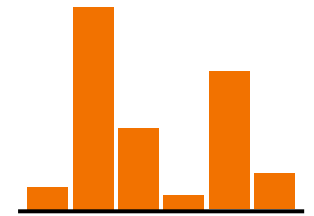
$$p_{\tau}(y|y_{<t}) = \frac{\exp \frac{p(y|y_{<t})}{\tau}}{\sum_{w \in V} \exp \frac{p(w|y_{<t})}{\tau}}$$



$p_2(y_t|y_{<t})$



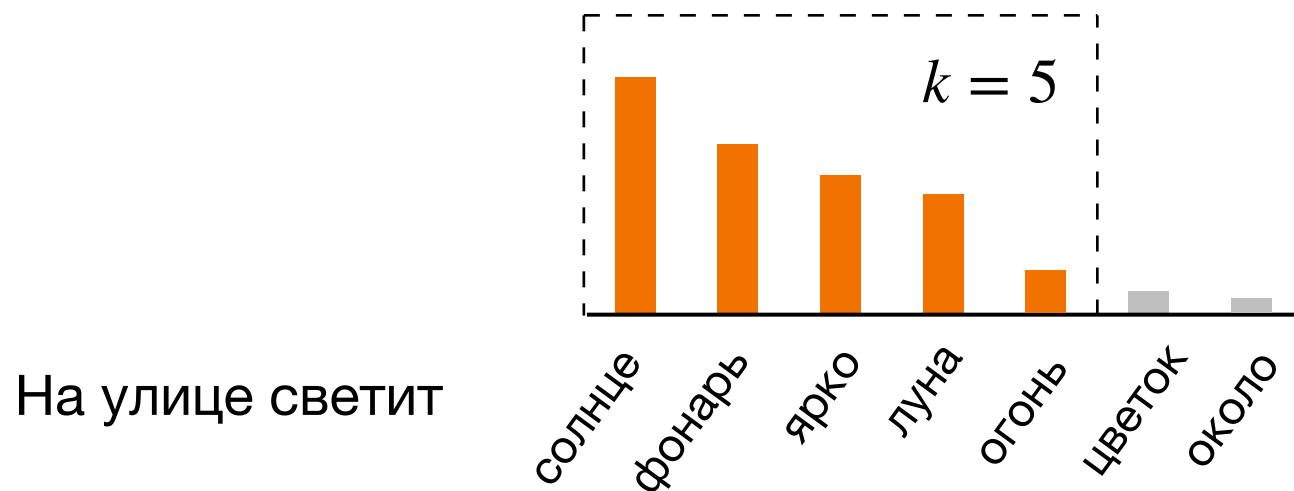
$p_1(y_t|y_{<t})$



$p_{0.5}(y_t|y_{<t})$

Тор-к семплирование

- Даже с температурой остается ненулевая генерация выдать случайный токен
- Оставим только k самых вероятных токенов и будем семплировать из них



Тор-р семплирование

Nucleus sampling

- Будем выбирать из минимального числа токенов, суммарная вероятность которых больше p .

$$\sum_{w \in V^{(p)}} p(w | y_{<t}) \geq p$$

$$|V^{(p)}| \rightarrow \min$$

Популярное значение для p – 0.9 или 0.95